

Documentazione Completa Progetto Blog Tecnico/Didattico

1. Introduzione
2. Incontro con il Cliente
3. Analisi dei Requisiti
4. Specifica dei requisiti (SRS)
5. Casi d'Uso
6. Diagramma delle Classi
7. Diagrammi di Sequenza
8. Diagramma delle Attività
9. Architettura Progetto
10. Testing
- 11.

1.Introduzione

Il presente documento raccoglie e descrive le attività di analisi e progettazione svolte per la realizzazione di una piattaforma web di tipo CMS (Content Management System) dedicata alla pubblicazione e gestione di contenuti tecnici e didattici. Il progetto nasce dall'esigenza di fornire uno strumento centralizzato che consenta la creazione, l'organizzazione e la consultazione di articoli, garantendo al tempo stesso una gestione strutturata degli utenti, delle categorie, dei tag e delle interazioni attraverso un sistema di commenti.

L'elaborato documenta il processo seguito durante lo sviluppo del sistema, partendo dall'analisi delle esigenze espresse dal committente fino alla definizione dell'architettura software, dei modelli UML e delle principali scelte progettuali. Il progetto prevede l'adozione di un'architettura basata sul pattern Model-View-Presenter (MVP), con frontend sviluppato mediante tecnologie web standard e backend accessibile tramite API REST.

Il progetto è stato sviluppato seguendo prevalentemente un modello a cascata, in cui le attività sono state organizzate in fasi sequenziali: raccolta dei requisiti, analisi, progettazione UML, definizione dell'architettura, implementazione, testing e documentazione finale. Tale approccio è risultato adeguato poiché i requisiti erano ben definiti fin dalle prime fasi e il dominio applicativo presentava una complessità contenuta. Pur mantenendo una struttura principalmente waterfall, alcune attività di sviluppo frontend e backend sono state svolte in parallelo, introducendo elementi di sviluppo incrementale.

La fase iniziale del lavoro è stata dedicata alla raccolta e alla formalizzazione dei requisiti funzionali e non funzionali. Per questo motivo, il documento si apre con il dialogo tra il committente e il team, durante il quale sono state identificate le necessità operative, gli obiettivi del sistema e le principali funzionalità richieste. Tale dialogo costituisce il punto di partenza per tutte le successive attività di analisi, progettazione e implementazione del software.

2. Incontro con il cliente

Partecipanti: Committente, Team di sviluppo.

Obiettivo dell'incontro: Raccogliere i requisiti per la realizzazione di una piattaforma web dedicata alla pubblicazione di contenuti tecnici e didattici.

Dialogo

Team: Buongiorno. Può spiegarci che tipo di applicazione desidera?

Cliente: Gestisco un piccolo portale di contenuti didattici e mi piacerebbe avere una piattaforma dove pubblicare articoli, guide e materiale informativo. Vorrei qualcosa di semplice da usare.

Team: Chi utilizzerà il sistema?

Cliente: Principalmente tre categorie di utenti:

- Visitatori che leggono i contenuti.
- Editor che scrivono articoli.
- Amministratori che gestiscono tutto il sito.

Team: Cosa dovrebbe poter fare un visitatore?

Cliente: Consultare gli articoli, leggerli e essere in grado di effettuare ricerche.

Team: Gli articoli dovrebbero essere commentabili?

Cliente: Sì, gli utenti registrati devono poter commentare gli articoli.

Team: I commenti devono essere pubblicati immediatamente?

Cliente: Preferirei poterli controllare prima della pubblicazione, per evitare spam o contenuti inappropriati.

Team: Cosa dovrebbe poter fare un Editor?

Cliente: Scrivere nuovi articoli e modificare quelli che ha creato personalmente; inoltre devono poter commentare gli articoli.

Team: E l'amministratore?

Cliente: Deve poter intervenire su qualsiasi contenuto, gestire gli utenti e moderare i commenti. Deve anche essere in grado di fare tutto quello che fanno gli altri ruoli.

Team: Come immagina l'organizzazione degli articoli?

Cliente: Vorrei che fossero divisi in categorie e che potessero avere più etichette per facilitarne la ricerca.

Team: Ha esigenze particolari riguardo alla grafica?

Cliente: Non mi interessa qualcosa di molto elaborato. Vorrei un'interfaccia chiara e intuitiva che funzioni bene; eventualmente adattabile agli smartphone.

Team: Come dovrebbe funzionare la ricerca?

Cliente: Gli utenti dovrebbero trovare facilmente un articolo inserendo parole chiave oppure selezionando una categoria o un'etichetta.

Team: Ha altre necessità?

Cliente: Mi piacerebbe poter vedere chi ha scritto ogni articolo e quando è stato pubblicato.

Team: Bene. Riassumendo, desidera una piattaforma per la gestione di contenuti didattici con articoli, categorie, tag, sistema di commenti moderati e diversi livelli di accesso per gli utenti.

Cliente: Esattamente.

3. Analisi dei requisiti

A seguito dell'incontro con il committente è stata effettuata un'attività di analisi finalizzata all'identificazione delle funzionalità necessarie e dei vincoli da rispettare nello sviluppo del sistema. Dalle informazioni raccolte è emersa la necessità di realizzare una piattaforma web per la pubblicazione e la gestione di contenuti tecnici e didattici, accessibile tramite browser e utilizzabile da utenti con differenti livelli di autorizzazione.

L'analisi ha evidenziato la presenza di tre principali categorie di utenti:

- Visitatore, interessato esclusivamente alla consultazione dei contenuti pubblicati;
- Editor, autorizzato alla creazione e gestione dei propri articoli nonché abilitato al commento degli articoli (propri o altrui);
- Amministratore, responsabile della gestione globale della piattaforma, degli utenti e delle attività di moderazione.

L'amministratore sarà inoltre in grado di effettuare tutte le operazioni garantite all'editor.

Per quanto riguarda la gestione dei contenuti, il sistema dovrà consentire la pubblicazione di articoli corredati da categorie e tag, in modo da facilitare l'organizzazione e la ricerca delle informazioni. È inoltre emersa la necessità di integrare un sistema di commenti soggetto a moderazione amministrativa e una funzionalità di ricerca basata su parole chiave e criteri di classificazione. Dal punto di vista tecnologico, il sistema dovrà essere sviluppato secondo un'architettura web client-server.

Attori Individuati

Attore	Responsabilità	Permessi
Visitatore	Consultazione e lettura dei contenuti didattici e tecnici messi a disposizione sulla piattaforma.	Lettura degli articoli; ricerca per parole chiave, categorie e tag.
Editor	Creazione e aggiornamento dei propri contenuti; partecipazione alle discussioni tramite commenti.	Tutti i permessi del Visitatore; creazione e modifica dei propri articoli; inserimento di commenti agli articoli.
Amministratore	Gestione globale della piattaforma, moderazione dei contenuti e controllo degli accessi.	Tutti i permessi dell'Editor; gestione utenti e ruoli; moderazione e approvazione dei commenti; gestione completa di categorie, tag ed eliminazione articoli.

Concetti principali del dominio

- Utente;
- Articolo;

- Categoria;
- Tag;
- Commento.

Servizi richiesti

- Pubblicazione articoli;
- Ricerca contenuti;
- Classificazione tramite categorie e tag;
- Sistema di commenti;
- Moderazione dei contenuti.

4. Specifica dei requisiti (SRS)

Requisiti Utente

I requisiti utente descrivono le esigenze espresse dal committente durante le attività di raccolta dei requisiti e rappresentano il punto di vista degli utilizzatori del sistema. Essi sono formulati in linguaggio naturale e descrivono le funzionalità attese senza entrare nei dettagli implementativi.

RU1 Il sistema deve consentire la pubblicazione e la consultazione di contenuti tecnici e didattici.

RU2 Gli articoli devono essere organizzati mediante categorie e tag per facilitarne la consultazione e la ricerca.

RU3 Gli utenti devono poter ricercare gli articoli attraverso parole chiave, categorie e tag.

RU4 Gli utenti registrati devono poter inserire commenti agli articoli pubblicati.

RU5 I commenti devono essere soggetti a moderazione prima della pubblicazione.

RU6 Gli editor devono poter creare e gestire i propri articoli.

RU7 Gli amministratori devono poter gestire utenti, contenuti e commenti dell'intera piattaforma.

RU8 L'interfaccia deve risultare semplice e intuitiva per gli utenti.

Requisiti di Sistema

I requisiti di sistema traducono le esigenze espresse dagli utenti in funzionalità che il software dovrà mettere a disposizione. Essi descrivono il comportamento atteso del sistema e costituiscono il collegamento tra i requisiti utente e la successiva formalizzazione delle specifiche software.

RS1 Il sistema deve consentire la visualizzazione degli articoli pubblicati.

RS2 Il sistema deve fornire strumenti di ricerca basati su parole chiave, categorie e tag.

RS3 Il sistema deve gestire tre tipologie di utenti: Visitatore, Editor e Amministratore.

RS4 Il sistema deve consentire agli editor la creazione, modifica ed eliminazione dei propri articoli.

RS5 Il sistema deve consentire l'inserimento di commenti da parte degli utenti autorizzati.

RS6 Il sistema deve permettere l'approvazione, la modifica e la rimozione dei commenti da parte degli amministratori.

RS7 Il sistema deve consentire la gestione di categorie e tag.

RS8 Il sistema deve mostrare per ogni articolo autore e data di pubblicazione.

RS9 Il sistema deve garantire differenti livelli di autorizzazione in base al ruolo dell'utente.

RS10 Il sistema deve consentire l'autenticazione degli utenti registrati.

Specifiche Software

A partire dai requisiti utente e dai requisiti di sistema precedentemente individuati, è stata effettuata la formalizzazione delle specifiche software. Le sezioni seguenti riportano i requisiti funzionali e non funzionali che costituiranno la base per la progettazione e la realizzazione del sistema.

Requisiti Funzionali

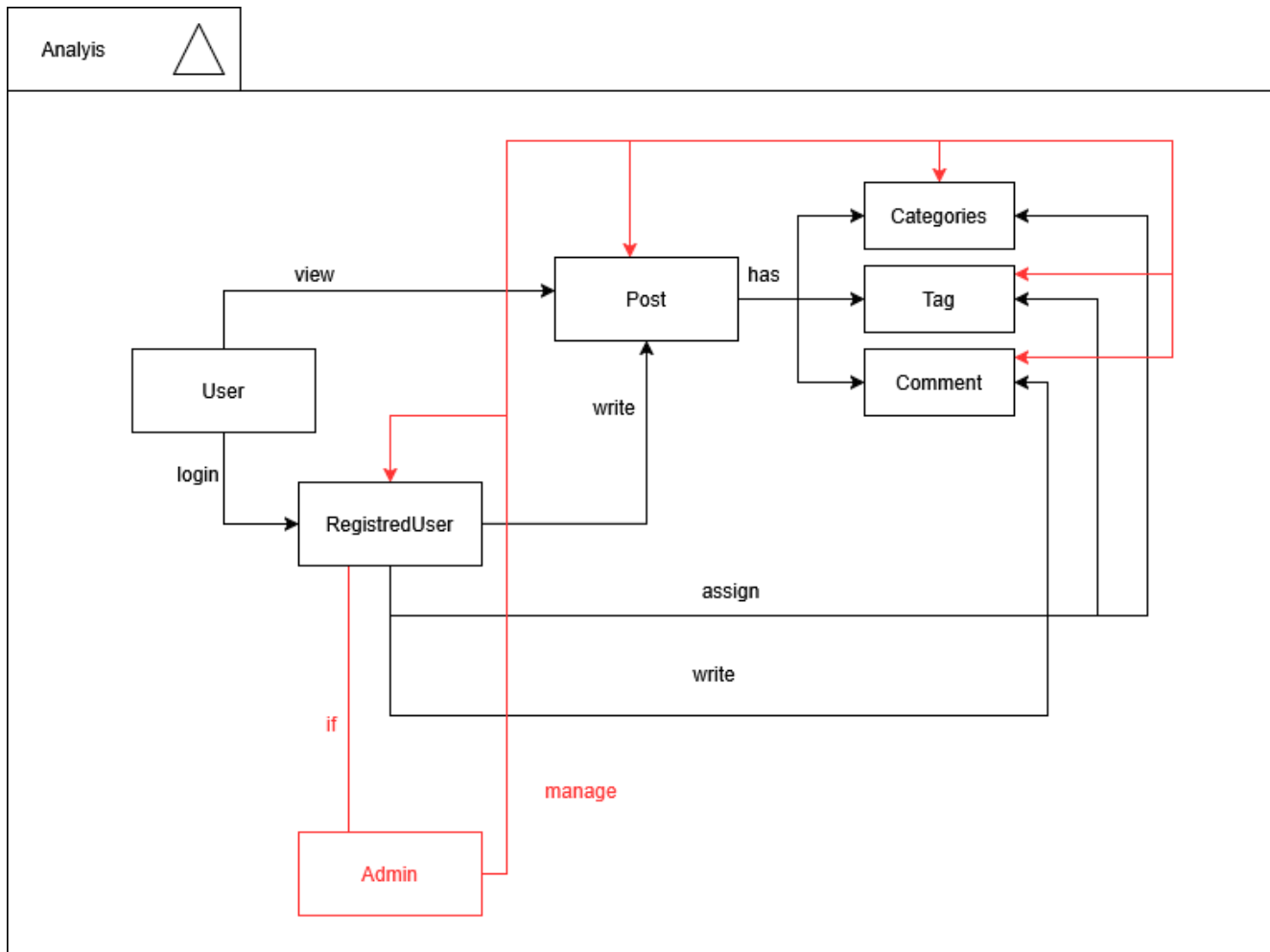
- RF1** Consultazione degli articoli
- RF2** Ricerca degli articoli per parole chiave
- RF3** Ricerca e navigazione per categoria e tag
- RF4** Creazione degli articoli
- RF5** Modifica dei propri articoli
- RF6** Eliminazione degli articoli secondo i permessi assegnati
- RF7** Gestione delle categorie (CRUD)
- RF8** Gestione dei tag (CRUD)
- RF9** Associazione di categorie agli articoli
- RF10** Associazione di uno o più tag agli articoli
- RF11** Inserimento dei commenti
- RF12** Moderazione dei commenti
- RF13** Visualizzazione di autore e data di pubblicazione
- RF14** Gestione utenti e ruoli
- RF15** Autenticazione degli utenti (login)

Requisiti Non Funzionali

- RNF1 - Semplicità d'uso** La piattaforma deve presentare un'interfaccia semplice e intuitiva, in modo da permettere agli utenti di consultare e gestire i contenuti senza particolari competenze tecniche.
- RNF2 - Facilità di consultazione dei contenuti** Gli articoli devono essere facilmente accessibili e leggibili da parte dei visitatori.
- RNF3 - Organizzazione delle informazioni** I contenuti devono poter essere organizzati attraverso categorie e tag per facilitarne la navigazione e l'individuazione.
- RNF4 - Efficacia della ricerca** Il sistema deve consentire agli utenti di individuare rapidamente gli articoli di interesse mediante funzionalità di ricerca e classificazione.
- RNF5 - Chiarezza dell'informazione** Le informazioni relative agli articoli, compresi autore, data di pubblicazione, categorie e tag, devono essere presentate in modo chiaro e facilmente comprensibile.
- RNF6 Compatibilità futura con dispositivi mobili** Il sistema deve prevedere eventuali aggiornamenti per integrare dispositivi mobili

Modello di Analisi

Il modello di analisi descrive i principali concetti del dominio applicativo individuati durante la fase di analisi dei requisiti e le relazioni che intercorrono tra essi. Tale modello costituisce la base per la successiva definizione dei casi d'uso e della struttura statica del sistema rappresentata dal diagramma delle classi.



Nel seguente modello di analisi sono rappresentati i principali concetti del dominio applicativo e le relazioni tra essi.

L'entità **User** rappresenta genericamente i tre attori del sistema (Visitatore, Editor e Amministratore), mentre **RegisteredUser** identifica gli utenti autenticati (Editor e Amministratore).

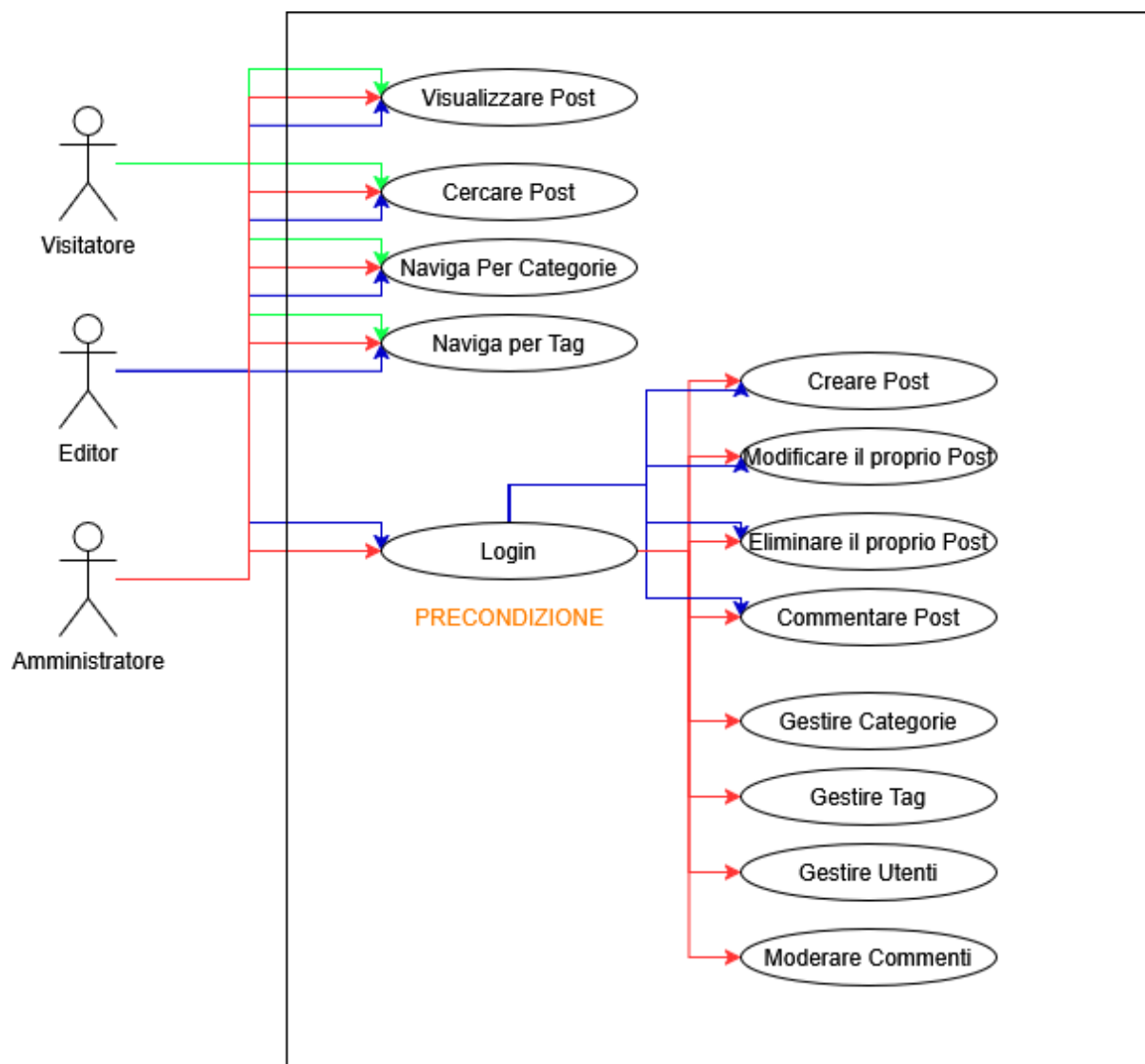
Per semplicità il modello approfondisce il ruolo di **Admin**, in quanto dotato del maggior numero di responsabilità e privilegi all'interno della piattaforma.

Conclusione

L'attività di analisi ha consentito di identificare e strutturare le esigenze del committente secondo differenti livelli di astrazione, distinguendo tra requisiti utente, requisiti di sistema e specifiche software. Questa classificazione garantisce una progressiva trasformazione delle richieste espresse in linguaggio naturale in requisiti formalizzati e verificabili, fornendo una solida base per le successive attività di progettazione, modellazione UML e implementazione del sistema.

5.Casi d'Uso

Il Diagramma dei Casi d'Uso descrive le interazioni tra gli attori e il sistema, evidenziando le funzionalità messe a disposizione dalla piattaforma e i diversi livelli di autorizzazione associati ai ruoli utente. Il diagramma è stato derivato dai requisiti raccolti durante la fase di analisi e consente di verificare la copertura delle funzionalità richieste dal committente. Gli attori del sistema sono organizzati secondo una gerarchia di generalizzazione: l'Editor eredita tutte le funzionalità del Visitatore, mentre l'Amministratore eredita sia le funzionalità dell'Editor sia quelle del Visitatore, aggiungendo ulteriori capacità di gestione e amministrazione della piattaforma.



Nel diagramma vengono evidenziati i tre attori del sistema: **Visitatore**, **Editor** e **Amministratore**. Il Visitatore può consultare e ricercare gli articoli pubblicati, navigando i contenuti attraverso categorie e tag; l'Editor, previa autenticazione, può creare, modificare ed eliminare i propri articoli oltre a inserire commenti oltre ad avere i permessi del Visitatore; l'Amministratore dispone di tutte le funzionalità dell'Editor e di ulteriori strumenti per la gestione degli utenti, delle categorie, dei tag e per la moderazione dei commenti. Il diagramma consente quindi di verificare la completezza dei requisiti funzionali individuati e costituisce la base per le successive attività di progettazione e modellazione del sistema.

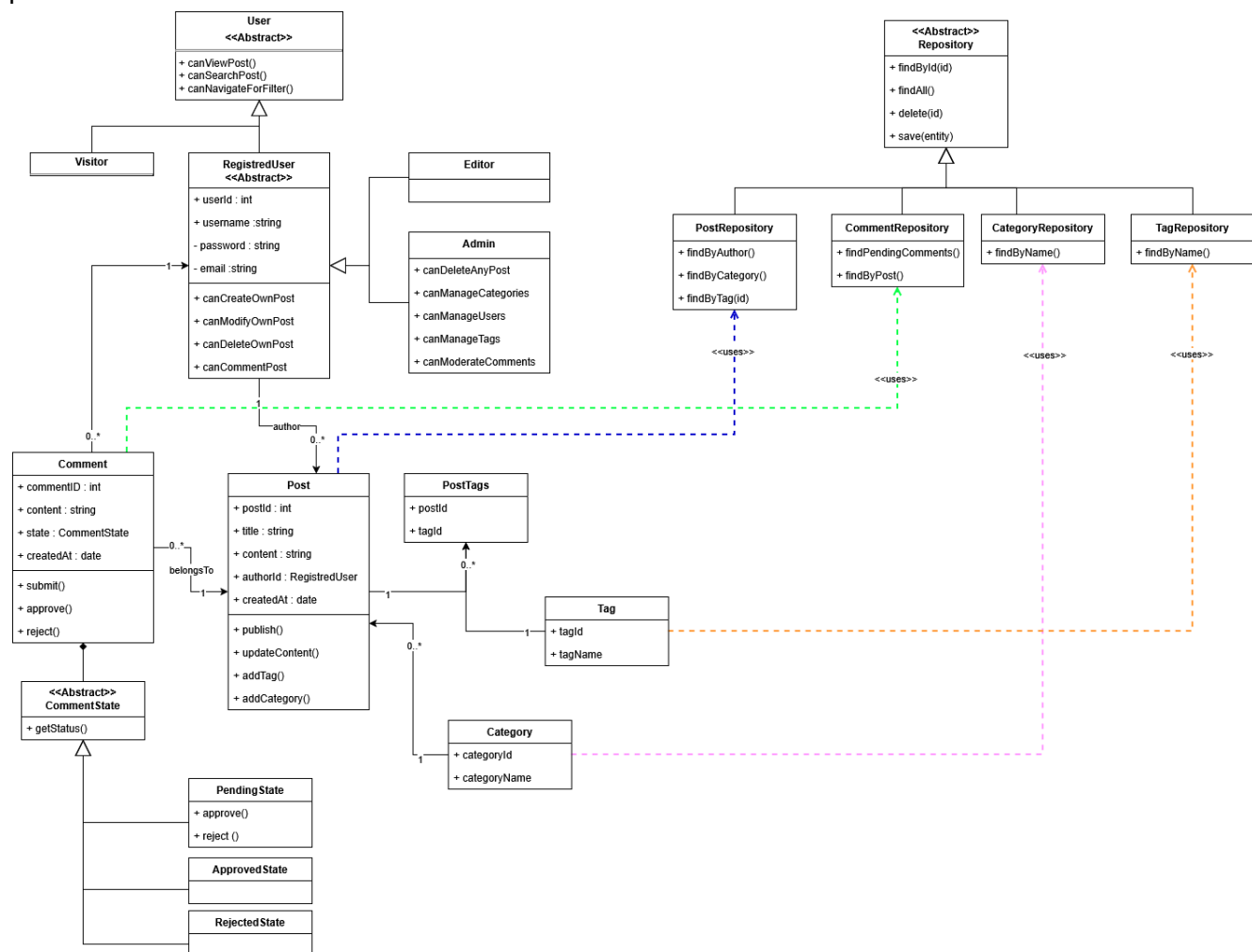
Matrice di Tracciabilità

La matrice di tracciabilità consente di verificare la copertura dei requisiti funzionali attraverso i casi d'uso individuati. Ogni requisito funzionale viene associato ad uno o più casi d'uso che ne rappresentano l'implementazione a livello di analisi, garantendo la coerenza tra fase di specifica dei requisiti e modellazione UML.

Requisito Funzionale	Caso d'Uso
RF1 Consultazione degli articoli	Visualizzare Post
RF2 Ricerca per parole chiave	Cercare Post
RF3 Ricerca e navigazione per categoria e tag	Naviga per Categoria, Naviga per Tag
RF4 Creazione degli articoli	Creare Post
RF5 Modifica dei propri articoli	Modificare il proprio Post
RF6 Eliminazione degli articoli secondo i permessi assegnati	Eliminare il proprio Post
RF7 Gestione delle categorie (CRUD)	Gestire Categorie
RF8 Gestione dei tag (CRUD)	Gestire Tag
RF9 Associazione di categorie agli articoli	Creare Post, Modificare il proprio Post
RF10 Associazione di uno o più tag agli articoli	Creare Post, Modificare il proprio Post
RF11 Inserimento dei commenti	Commentare Post
RF12 Moderazione dei commenti	Moderare Commenti
RF13 Visualizzazione di autore e data di pubblicazione	Visualizzare Post
RF14 Gestione utenti e ruoli	Gestire Utenti
RF15 Autenticazione degli utenti	Login

6. Diagramma delle Classi

Il diagramma delle classi rappresenta la struttura statica del sistema e descrive le principali entità individuate durante le fasi di analisi dei requisiti e modellazione dei casi d'uso. Attraverso tale diagramma vengono definiti gli attributi, le operazioni e le relazioni che caratterizzano gli oggetti del dominio applicativo, fornendo una visione dettagliata dell'organizzazione interna del software. Il diagramma costituisce il punto di collegamento tra l'analisi del problema e la successiva progettazione dell'architettura, consentendo di formalizzare i concetti individuati nelle fasi precedenti in una struttura coerente e facilmente implementabile. Le classi modellate rappresentano gli utenti del sistema, gli articoli, le categorie, i tag, i commenti e i componenti necessari alla gestione e persistenza dei dati.



Il diagramma delle classi è stato progettato per rappresentare i principali concetti del dominio applicativo e le relative relazioni. Alla base della gerarchia degli utenti è presente la classe astratta **User**, dalla quale derivano le diverse tipologie di utenti del sistema. Il **Visitatore** dispone esclusivamente delle funzionalità di consultazione e ricerca dei contenuti, mentre le classi **Editor** e **Admin** estendono le capacità degli utenti registrati introducendo funzionalità di gestione dei contenuti e amministrazione della piattaforma.

La classe **Post** rappresenta l'elemento centrale del sistema e contiene le informazioni relative agli articoli pubblicati. Ogni articolo è associato ad un autore e può essere classificato mediante una categoria e uno o più tag. Tale struttura consente di organizzare i contenuti in modo efficace e di supportare le funzionalità di ricerca e navigazione richieste dal committente.

Le classi **Category** e **Tag** permettono la classificazione degli articoli secondo differenti criteri. In particolare, la relazione molti-a-molti tra post e tag viene gestita attraverso l'entità **PostTags**, che consente l'associazione di più etichette ad uno stesso articolo e viceversa.

La classe **Comment** modella il sistema di commenti previsto tra i requisiti della piattaforma. Ogni commento è associato ad un articolo e possiede uno stato che ne determina il livello di approvazione, consentendo agli amministratori di effettuare le attività di moderazione prima della pubblicazione. Per la gestione del ciclo di vita dei commenti è stato adottato lo **State Pattern**, attraverso il quale il comportamento del commento varia in funzione del suo stato corrente. Gli stati possibili sono rappresentati dalle classi **PendingState**, **ApprovedState** e **RejectedState**, specializzazioni della classe astratta **CommentState**. Tale soluzione consente di separare la logica di moderazione dalla classe **Comment** e di rendere più semplice l'introduzione di ulteriori stati o regole di transizione in future evoluzioni del sistema.

Infine, il diagramma introduce un insieme di classi dedicate alla persistenza dei dati attraverso il **Repository Pattern**. Tali componenti hanno il compito di isolare la logica di accesso ai dati dalle entità del dominio, migliorando la manutenibilità e l'organizzazione complessiva del sistema. Insieme allo **State Pattern** utilizzato per la gestione dei commenti, questa scelta progettuale favorisce la modularità del software, la separazione delle responsabilità e la futura estendibilità dell'applicazione.

Conclusione

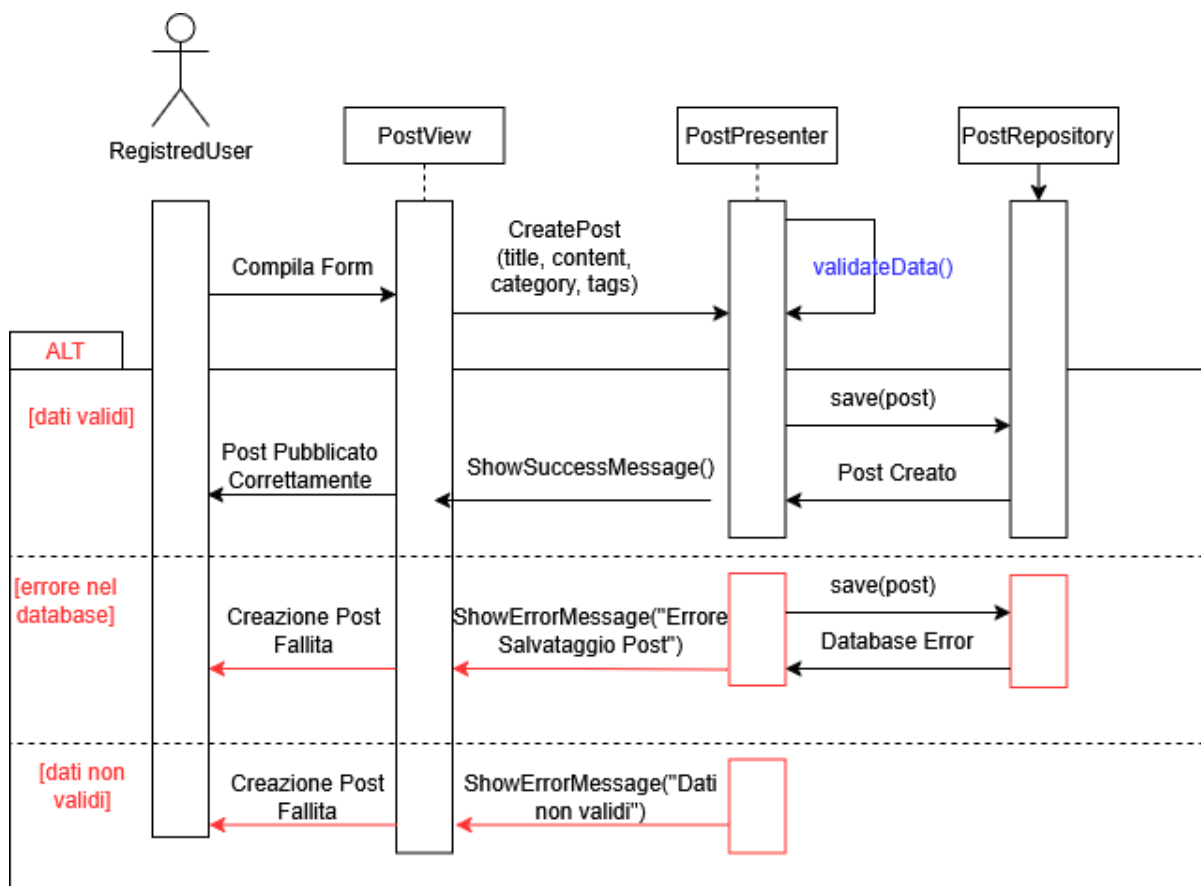
Il diagramma delle classi formalizza la struttura dati del sistema e definisce le relazioni necessarie per supportare i requisiti funzionali individuati nelle fasi precedenti. Esso rappresenta il modello statico di riferimento per la successiva implementazione delle componenti software e per la definizione dei diagrammi comportamentali, come i diagrammi di sequenza e di attività.

7.Diagrammi di Sequenza

I diagrammi di sequenza illustrano il flusso delle interazioni tra gli oggetti del sistema durante l'esecuzione di uno specifico caso d'uso. Attraverso la rappresentazione temporale dei messaggi scambiati tra i componenti, essi consentono di comprendere il comportamento dinamico dell'applicazione e di verificare come i requisiti funzionali vengono soddisfatti dal sistema.

Diagramma di Sequenza - Creazione Post

Il diagramma di sequenza relativo alla creazione di un post descrive il flusso di interazioni che avviene tra gli elementi del sistema quando un utente autorizzato pubblica un nuovo articolo. Questo diagramma consente di rappresentare il comportamento dinamico del sistema, evidenziando l'ordine temporale dei messaggi scambiati tra interfaccia utente, componenti di controllo e componenti di accesso ai dati. L'obiettivo è mostrare come il requisito funzionale relativo alla creazione degli articoli venga realizzato attraverso la collaborazione tra le diverse componenti software del sistema.



Il processo di creazione di un post inizia quando un utente autenticato compila il modulo dedicato all'inserimento di un nuovo articolo, specificando le informazioni richieste, quali titolo, contenuto, categoria e tag associati. Tali dati vengono raccolti dall'interfaccia utente e inviati al componente incaricato della gestione della logica applicativa.

Il componente di controllo riceve le informazioni inserite dall'utente ed esegue le opportune verifiche di validazione. In questa fase viene controllata la correttezza e la completezza dei dati forniti, così da garantire che il contenuto possa essere pubblicato senza errori o incongruenze.

Se la validazione ha esito positivo, il sistema procede con il salvataggio del nuovo articolo attraverso il componente responsabile della persistenza dei dati. Una volta completata con successo l'operazione di memorizzazione, il sistema restituisce un messaggio di conferma all'utente e il post viene pubblicato.

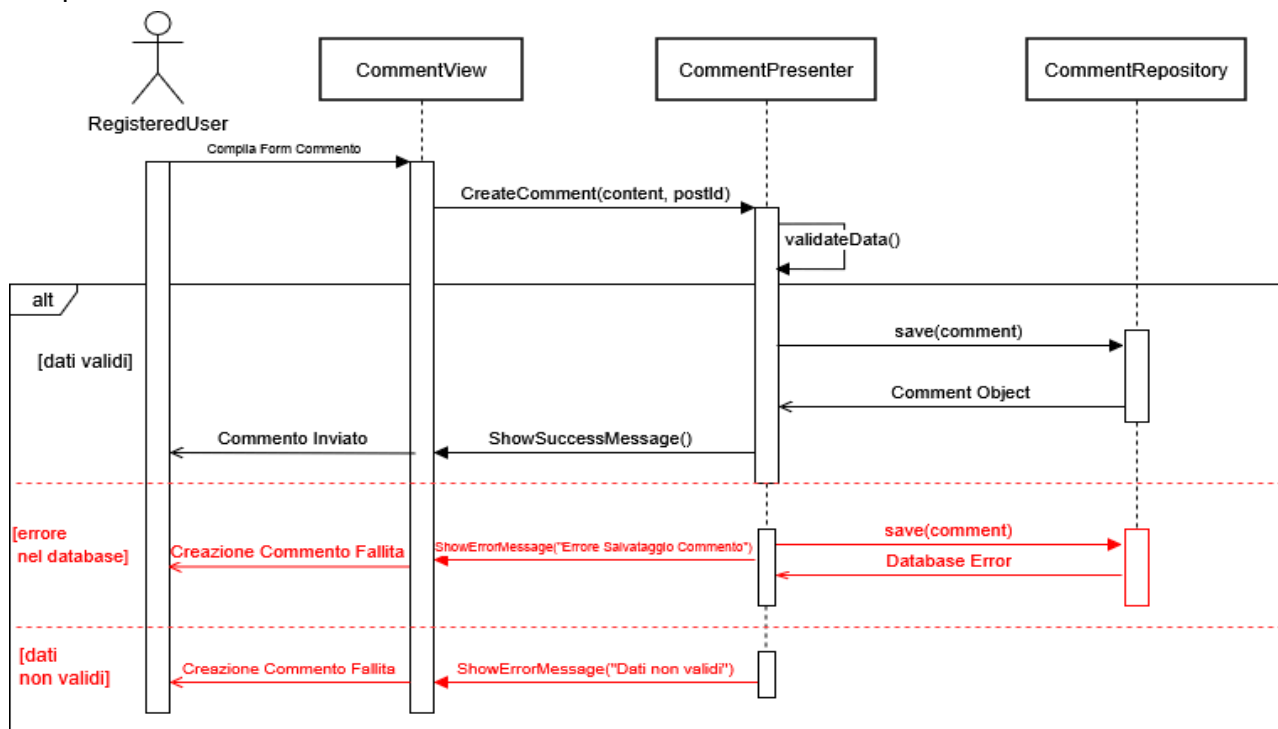
Nel caso in cui i dati inseriti non risultino validi, il processo viene interrotto e il sistema notifica all'utente la presenza dell'errore, richiedendo la correzione delle informazioni fornite prima di consentire una nuova operazione di pubblicazione.

Il diagramma evidenzia inoltre la chiara separazione delle responsabilità tra i diversi componenti del sistema: l'interfaccia utente si occupa della raccolta e visualizzazione delle informazioni, il componente di controllo gestisce la logica applicativa e le verifiche necessarie, mentre il componente dedicato alla persistenza dei dati si occupa esclusivamente del salvataggio delle informazioni. Questa organizzazione risulta coerente con l'architettura adottata dal progetto e favorisce la modularità e la manutenibilità del software.

Il diagramma di sequenza della creazione di un post mostra come il sistema realizzi una delle funzionalità centrali della piattaforma, garantendo il corretto inserimento e la successiva pubblicazione degli articoli. La modellazione del flusso di interazione fornisce una rappresentazione dettagliata del comportamento dinamico del sistema e costituisce un utile riferimento per la successiva fase di implementazione.

Diagramma di Sequenza - Inserimento Commento

Il seguente diagramma di sequenza descrive il processo di inserimento di un commento da parte di un utente registrato all'interno della piattaforma. Il diagramma mostra l'interazione tra i componenti del pattern Model-View-Presenter (MVP) coinvolti nella gestione dei commenti, evidenziando sia il flusso di esecuzione corretto sia le possibili condizioni di errore dovute a dati non validi o problemi durante il salvataggio delle informazioni. La sequenza garantisce che il commento venga sottoposto a validazione prima della persistenza nel sistema e che l'utente riceva un opportuno riscontro sull'esito dell'operazione.



Il processo ha inizio quando l'utente registrato compila il modulo di inserimento del commento attraverso la `CommentView`. Una volta completata la compilazione, i dati del commento vengono inviati al `CommentPresenter`, che assume il ruolo di coordinatore del flusso applicativo secondo il pattern MVP.

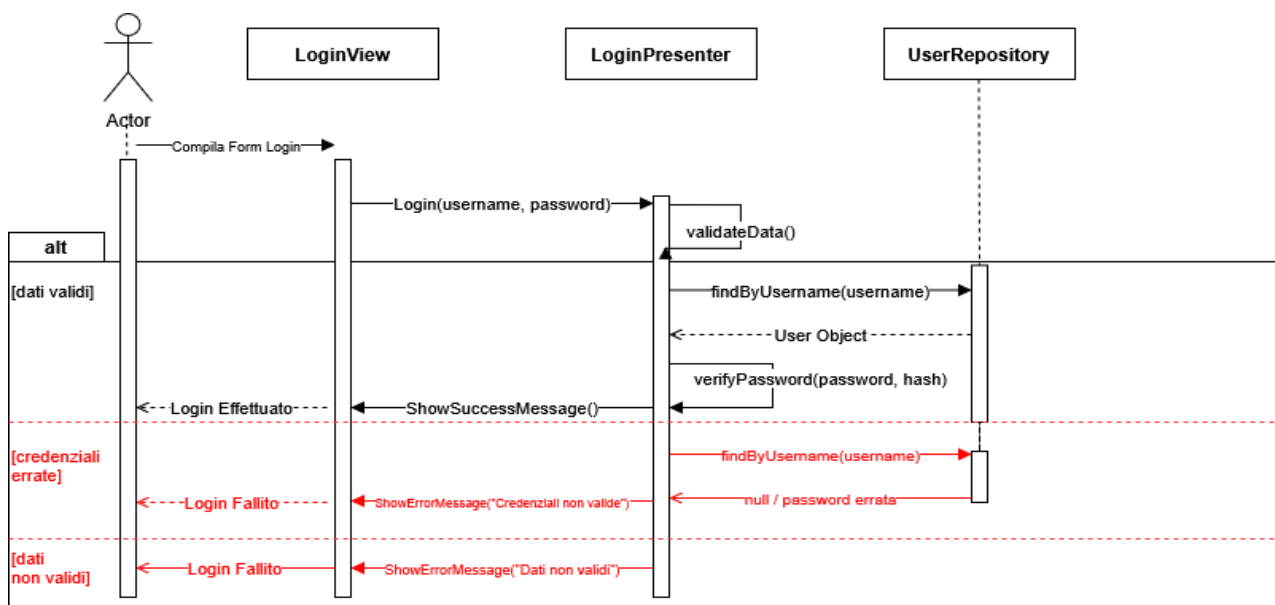
Il `CommentPresenter` effettua una fase preliminare di validazione delle informazioni ricevute verificando la correttezza dei dati inseriti dall'utente. Se la validazione viene superata con successo, il presenter procede alla richiesta di salvataggio del commento tramite il `CommentRepository`. Nel caso in cui l'operazione di persistenza venga completata correttamente, il repository restituisce l'oggetto commento appena creato. Il presenter informa quindi la vista dell'avvenuto inserimento mediante un messaggio di conferma, che viene mostrato all'utente sotto forma di notifica di successo. Il diagramma evidenzia inoltre due possibili scenari alternativi di errore. Nel primo caso, qualora i dati inseriti non rispettino i criteri di validazione previsti dal sistema, il `CommentPresenter` interrompe il flusso e comunica immediatamente alla vista la presenza di dati non validi, impedendo il salvataggio del commento.

Nel secondo scenario, i dati risultano formalmente corretti ma si verifica un problema durante l'operazione di persistenza nel repository. In tale situazione il `CommentRepository` restituisce un errore al presenter, che provvede a notificare la vista mostrando un messaggio di errore relativo al mancato salvataggio del commento.

L'utilizzo del pattern MVP consente di mantenere separata la logica di presentazione dalla logica applicativa e dall'accesso ai dati, garantendo una maggiore modularità e facilitando le attività di manutenzione e testing del sistema.

Diagramma di Sequenza - Login

Il diagramma di sequenza relativo al processo di login descrive le interazioni che avvengono tra i componenti del sistema durante l'autenticazione di un utente. Attraverso la rappresentazione temporale dei messaggi scambiati tra interfaccia utente, componenti applicative e livello di persistenza, il diagramma mostra come il sistema verifichi le credenziali fornite e consenta l'accesso alle funzionalità riservate agli utenti autenticati. Tale processo risulta fondamentale per la gestione dei ruoli e dei permessi previsti all'interno della piattaforma CMS.



Il processo di autenticazione ha inizio quando l'utente inserisce le proprie credenziali nella schermata di accesso. Le informazioni fornite vengono raccolte dall'interfaccia utente e inoltrate al componente responsabile della gestione dell'autenticazione. Il Presenter coordina la richiesta e provvede a trasmetterla al servizio dedicato alla verifica delle credenziali.

Successivamente il backend riceve la richiesta di autenticazione ed effettua i controlli necessari confrontando le informazioni fornite dall'utente con i dati memorizzati nel database. Durante questa fase vengono inoltre verificati il ruolo associato all'account e i relativi permessi di accesso.

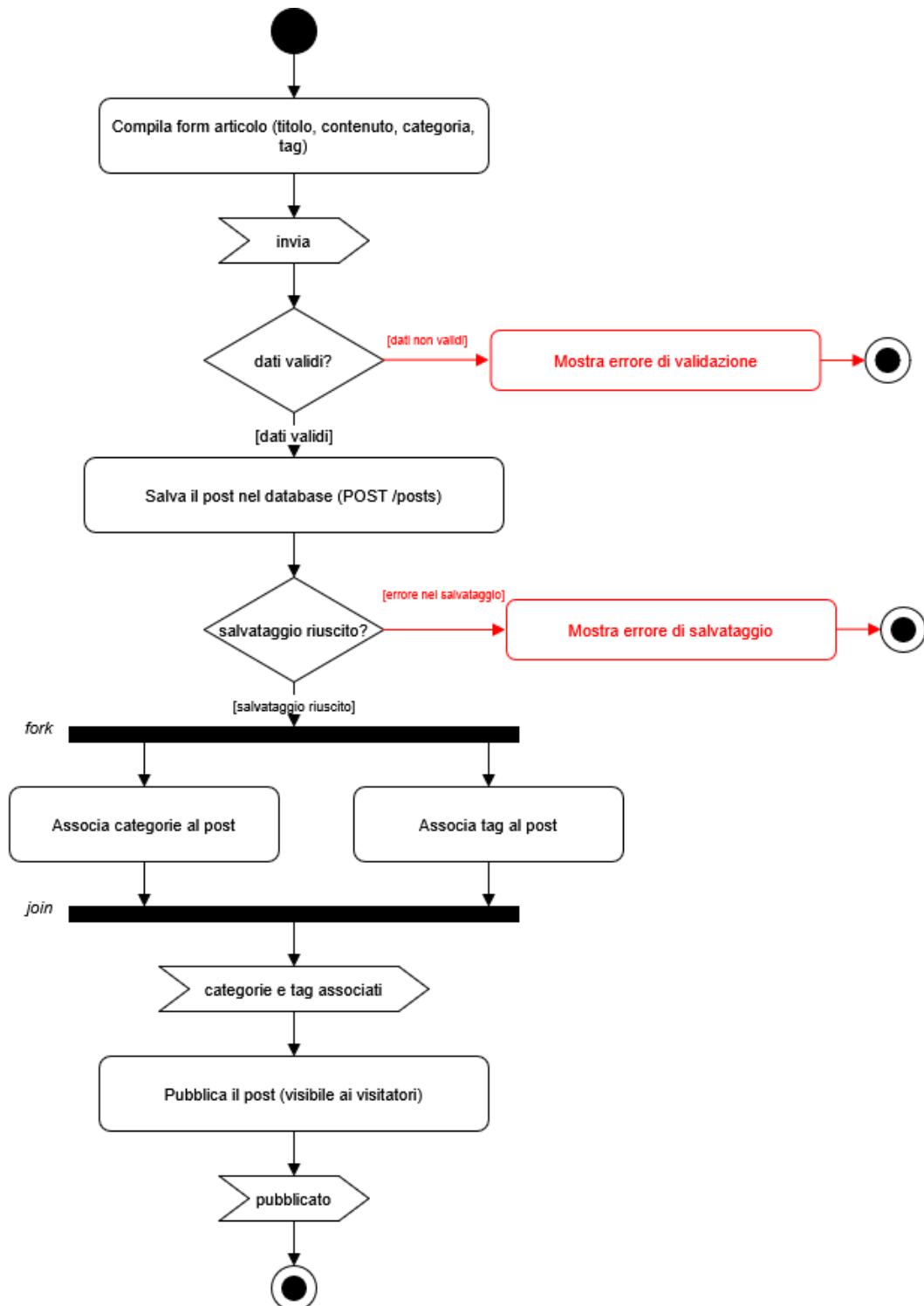
Se le credenziali risultano valide, il sistema conferma l'autenticazione e restituisce all'utente l'accesso alle funzionalità consentite dal proprio ruolo. In caso contrario, viene generato un messaggio di errore che informa l'utente del mancato accesso e richiede l'inserimento di credenziali corrette.

Il diagramma mette inoltre in evidenza la collaborazione tra i diversi livelli dell'architettura. La View si occupa esclusivamente della raccolta e visualizzazione delle informazioni, il Presenter gestisce il flusso applicativo e i servizi di autenticazione, mentre il backend e il livello di persistenza eseguono le verifiche necessarie sull'account dell'utente. Questa organizzazione risulta coerente con l'architettura MVP adottata nel progetto e favorisce una chiara separazione delle responsabilità tra i componenti del sistema.

Il diagramma di sequenza del login mostra come il sistema realizzi il processo di autenticazione degli utenti garantendo il controllo degli accessi e l'applicazione dei permessi associati ai diversi ruoli della piattaforma. La modellazione del flusso di autenticazione consente di comprendere il comportamento dinamico del sistema e di verificare la corretta implementazione dei requisiti relativi alla sicurezza e alla gestione degli utenti.

8. Diagramma delle Attività

Il diagramma delle attività rappresenta il flusso operativo necessario per la realizzazione di una determinata funzionalità del sistema, evidenziando le azioni eseguite, i punti decisionali e i possibili percorsi alternativi. A differenza dei diagrammi di sequenza, che descrivono le interazioni tra componenti software, il diagramma delle attività si concentra sul processo logico e sul susseguirsi delle operazioni necessarie al raggiungimento di un obiettivo. Nel caso in esame viene modellato il processo di creazione e pubblicazione di un articolo all'interno della piattaforma CMS.



Il flusso ha inizio con la compilazione del modulo di creazione dell'articolo da parte dell'utente autorizzato, che inserisce le informazioni necessarie quali titolo, contenuto, categoria e tag. Una volta completata la compilazione, i dati vengono inviati al sistema per la validazione.

La prima decisione riguarda la correttezza delle informazioni inserite. Qualora i dati non risultino validi, il processo viene interrotto e il sistema mostra un messaggio di errore che richiede la correzione delle informazioni fornite. In caso di esito positivo, il sistema procede con il salvataggio dell'articolo nel database.

Successivamente viene verificato l'esito dell'operazione di persistenza. Se il salvataggio non viene completato correttamente, il processo termina con la visualizzazione di un messaggio di errore relativo al salvataggio dei dati. Se invece l'operazione ha successo, il flusso prosegue con l'associazione delle informazioni di classificazione dell'articolo.

Il diagramma evidenzia inoltre una fase di elaborazione parallela mediante costrutti di fork e join.

Dopo il salvataggio dell'articolo vengono eseguite contemporaneamente l'associazione delle categorie e l'associazione dei tag. Terminata l'esecuzione di entrambe le attività, il sistema sincronizza il flusso e prosegue alla fase successiva.

Completata la classificazione del contenuto, l'articolo viene pubblicato e reso disponibile ai visitatori della piattaforma. Il processo termina quindi con la conferma dell'avvenuta pubblicazione, concludendo con successo l'intero flusso operativo.

Il diagramma delle attività descrive in maniera dettagliata il flusso operativo della pubblicazione di un articolo, evidenziando le verifiche, le condizioni di errore e le attività eseguite in parallelo. Tale rappresentazione consente di comprendere il comportamento logico del sistema e di verificare la corretta realizzazione dei requisiti funzionali relativi alla creazione e gestione dei contenuti.

9.Architettura Progetto

L'architettura del sistema definisce l'organizzazione delle principali componenti software e le modalità con cui esse collaborano per soddisfare i requisiti individuati durante la fase di analisi. L'obiettivo principale è garantire una chiara separazione delle responsabilità, favorendo manutenibilità, modularità ed estendibilità dell'applicazione. Il sistema è stato progettato come una piattaforma web dedicata alla pubblicazione e gestione di contenuti tecnici e didattici, articolata in componenti specializzate per la gestione degli utenti, degli articoli, delle categorie, dei tag e dei commenti. L'applicazione adotta un'architettura client-server, nella quale il frontend si occupa dell'interazione con l'utente mentre il backend gestisce la logica applicativa, le regole di business e l'accesso ai dati persistenti. La comunicazione tra i due livelli avviene mediante API REST che consentono lo scambio di informazioni in formato JSON.

Per migliorare la qualità del software e ridurre l'accoppiamento tra i componenti, l'architettura integra inoltre specifici pattern progettuali e soluzioni organizzative che consentono di separare la logica applicativa dalla gestione dell'interfaccia utente e della persistenza dei dati. Le sezioni seguenti descrivono nel dettaglio le scelte architetturali adottate e le motivazioni che hanno portato alla loro selezione.

Architettura Client-Server

Il sistema è stato progettato secondo un'architettura **Client-Server**, un modello ampiamente utilizzato nello sviluppo di applicazioni web che consente di separare la gestione dell'interfaccia utente dalla logica applicativa e dalla persistenza dei dati. In tale architettura il client e il server operano come componenti distinti e collaborano attraverso una rete mediante lo scambio di richieste e risposte. Nel progetto sviluppato, il **client** è rappresentato dall'applicazione frontend realizzata mediante HTML, CSS e JavaScript. Il suo compito è fornire l'interfaccia grafica agli utenti, raccogliere gli input provenienti dall'interazione con il sistema e visualizzare i risultati restituiti dal backend. Le funzionalità di presentazione vengono organizzate secondo il pattern MVP, garantendo una chiara separazione tra visualizzazione dei dati e logica applicativa.

Il **server** è costituito dall'applicazione backend sviluppata in PHP. Tale componente è responsabile dell'elaborazione delle richieste provenienti dal client, dell'applicazione delle regole di business e della gestione dei dati persistenti. Il backend implementa inoltre i meccanismi di autenticazione, autorizzazione e controllo degli accessi in funzione del ruolo dell'utente.

La comunicazione tra client e server avviene esclusivamente tramite **API REST**, che permettono lo scambio di dati in formato JSON. Quando un utente esegue un'operazione sulla piattaforma, ad esempio la consultazione di un articolo o la creazione di un nuovo contenuto, il frontend invia una richiesta al backend. Quest'ultimo elabora la richiesta, accede ai dati necessari e restituisce una risposta che viene successivamente visualizzata all'utente attraverso l'interfaccia grafica.

L'accesso alle informazioni archiviate viene gestito mediante il **Repository Pattern**, che introduce un livello di astrazione tra la logica applicativa e il database. Questa soluzione consente di centralizzare le operazioni di persistenza dei dati e di ridurre l'accoppiamento tra le diverse componenti del sistema.

L'adozione dell'architettura Client-Server offre numerosi vantaggi, tra cui una maggiore modularità del software, una migliore manutenibilità del codice e la possibilità di sviluppare ed evolvere frontend e backend in modo relativamente indipendente. Inoltre, tale struttura risulta particolarmente adatta alle esigenze della piattaforma CMS, caratterizzata dalla presenza di differenti tipologie di utenti, funzionalità di gestione dei contenuti e servizi di autenticazione e moderazione.

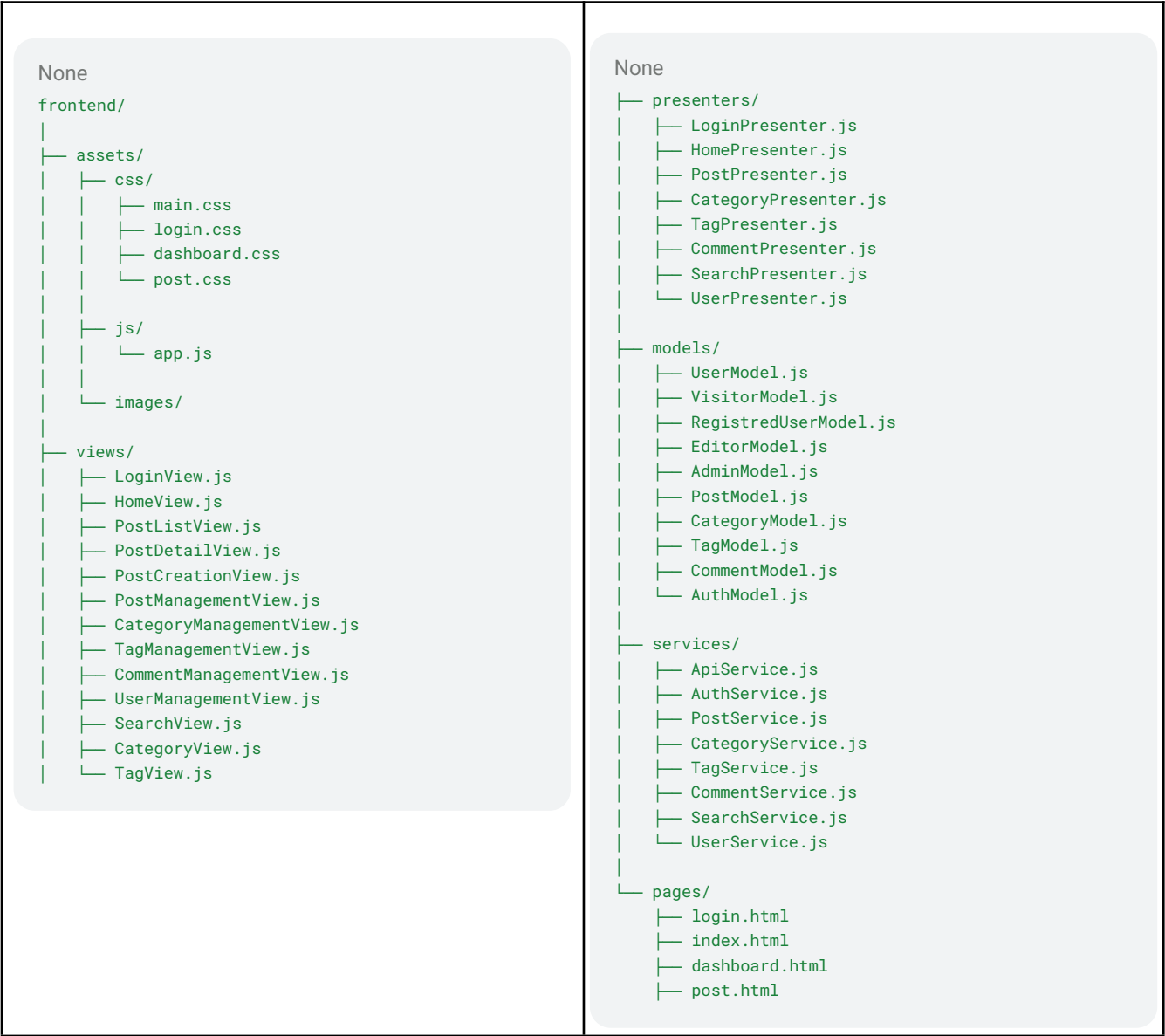
Architettura Frontend

Il frontend è stato progettato secondo il pattern architetturale Model-View-Presenter (MVP), che consente di separare la presentazione dei dati dalla logica applicativa e dalla gestione dello stato dell'applicazione. Tale approccio favorisce la modularità del sistema e semplifica la manutenzione del codice.

Nel pattern MVP la View gestisce l'interfaccia utente, il Presenter coordina il flusso applicativo e il Model rappresenta i dati del dominio. Questa separazione delle responsabilità riduce l'accoppiamento tra i componenti e migliora la testabilità del software.

Componenti principali del sistema

Il frontend adotta il pattern architetturale Model-View-Presenter (MVP), mentre il backend è organizzato secondo una suddivisione in Controller, Services e Repository. Tale approccio consente di separare la logica applicativa dalla rappresentazione dei dati e dall'interfaccia utente, favorendo modularità, manutenibilità e testabilità del software.





Organizzazione dei Componenti Frontend

Il frontend adotta il pattern architetturale Model-View-Presenter (MVP), mentre il backend è organizzato secondo una suddivisione in Controller, Services e Repository

View

Le View gestiscono esclusivamente la presentazione dei dati e l'interazione con l'utente. Ogni funzionalità principale del sistema dispone di una vista dedicata, tra cui:

- LoginView
- PostListView
- PostDetailView
- PostCreationView
- CategoryManagementView
- TagManagementView
- UserManagementView

Le View non contengono logica di business ma delegano tutte le operazioni ai Presenter.

Presenter

I Presenter rappresentano il punto centrale del flusso applicativo e coordinano l'interazione tra View, Model e Services.

I principali Presenter sono:

- LoginPresenter: gestione autenticazione.
- PostPresenter: gestione creazione, modifica e pubblicazione articoli.
- CategoryPresenter: gestione categorie.
- TagPresenter: gestione tag.
- CommentPresenter: gestione commenti.
- SearchPresenter: gestione funzionalità di ricerca.
- UserPresenter: gestione utenti.

Model

I Model rappresentano le entità del dominio applicativo:

- UserModel

- PostModel
- CategoryModel
- TagModel
- CommentModel
- AuthModel

Essi mantengono la struttura dei dati indipendente dalla presentazione.

Service Layer

I Service fungono da intermediari tra frontend e backend attraverso API REST.

Tra i principali servizi troviamo:

- ApiService
- AuthService
- PostService
- CategoryService
- TagService
- CommentService
- SearchService
- UserService

Ogni servizio encapsula le chiamate HTTP e la gestione delle risposte provenienti dal backend.

Flusso di Interazione nel Frontend

In seguito a un'azione effettuata dall'utente, la View inoltra la richiesta al Presenter, il quale esegue le opportune verifiche e interagisce con il Model e con i servizi applicativi. Una volta ottenuti i dati necessari, il Presenter aggiorna la View che provvede alla visualizzazione del risultato. Tale meccanismo consente di ridurre l'accoppiamento tra interfaccia utente e logica applicativa, favorendo modularità, testabilità e manutenibilità del sistema.

L'adozione del pattern Model-View-Presenter consente di organizzare il frontend secondo una struttura modulare basata sulla separazione delle responsabilità. Grazie alla distinzione tra presentazione, logica applicativa e gestione dei dati, il sistema risulta più facilmente manutenibile, estendibile e coerente con i requisiti individuati durante le fasi di analisi e progettazione

Architettura Backend

L'architettura backend del Blog Tecnico-Didattico è progettata per gestire la logica applicativa, l'accesso ai dati e l'esposizione delle API REST necessarie alla comunicazione con il frontend. Il sistema è implementato in PHP e segue una organizzazione modulare che favorisce manutenibilità, riuso del codice ed estendibilità. Inoltre, la soluzione rispetta i vincoli progettuali che richiedono l'utilizzo di PHP, API REST e pattern MVP.

Componenti principali del sistema

La struttura principale del backend è organizzata nelle seguenti directory:

None

backend/

|

├─ api/

|

└─ index.php

|

├─ controllers/

|

└─ AuthController.php

|

└─ PostController.php

|

└─ CategoryController.php

|

└─ TagController.php

|

└─ CommentController.php

|

└─ UserController.php

|

├─ services/

|

└─ AuthService.php

|

└─ PostService.php

|

└─ CategoryService.php

|

└─ TagService.php

|

└─ CommentService.php

|

└─ UserService.php

|

├─ repositories/

|

└─ Repository.php

|

└─ UserRepository.php

|

└─ PostRepository.php

|

└─ CategoryRepository.php

|

└─ TagRepository.php

|

└─ CommentRepository.php

|

├─ models/

|

└─ User.php

|

└─ Visitor.php

|

└─ RegisteredUser.php

|

└─ Editor.php

|

└─ Admin.php

|

└─ Post.php

|

└─ Category.php

|

└─ Tag.php

|

└─ PostTags.php

|

└─ Comment.php

|

└─ CommentStatus.php

|

└─ config/

|

└─ Database.php

Questa organizzazione separa chiaramente la logica applicativa dalla persistenza dei dati e dalla gestione delle richieste HTTP.

Controller

I controller rappresentano il punto di accesso alle API REST del sistema. Essi ricevono le richieste provenienti dal frontend, validano i dati ricevuti e delegano l'esecuzione delle operazioni ai servizi applicativi. Sono presenti controller dedicati alla gestione dell'autenticazione, degli articoli, delle categorie, dei tag, dei commenti e degli utenti.

Service

I servizi implementano la logica di business dell'applicazione. Ogni servizio coordina le operazioni necessarie per completare una funzionalità e interagisce con i repository per il recupero e il salvataggio dei dati. Questo livello consente di mantenere i controller leggeri e focalizzati esclusivamente sulla gestione delle richieste HTTP.

Repository

Il layer Repository gestisce l'accesso al database e incapsula tutte le operazioni CRUD. L'utilizzo del Repository Pattern garantisce indipendenza tra logica applicativa e persistenza dei dati, migliorando la manutenibilità del sistema. Tutti i repository specializzati derivano da una classe base comune che fornisce le funzionalità condivise.

Model

I model rappresentano le entità fondamentali del dominio applicativo. Tra queste troviamo utenti, articoli, commenti, categorie e tag. La gerarchia degli utenti rispecchia quanto definito nel diagramma delle classi, distinguendo le figure di Visitatore, Editor e Amministratore.

Gestione dei Ruoli

La gestione dei ruoli è basata sulla gerarchia di classi definita nel modello a oggetti. Le classi Visitor, Editor e Admin derivano dalla classe astratta User e rappresentano i differenti livelli di autorizzazione previsti dal sistema. Tale soluzione consente di associare comportamenti e permessi specifici a ciascun ruolo mantenendo una struttura semplice e coerente con il modello del dominio.

Database e Persistenza

L'accesso al database è centralizzato attraverso il componente Database, che gestisce la connessione e fornisce i servizi necessari ai repository. Questa soluzione evita duplicazioni di codice e garantisce una gestione uniforme delle operazioni di persistenza.

Flusso di Interazione nel Backend

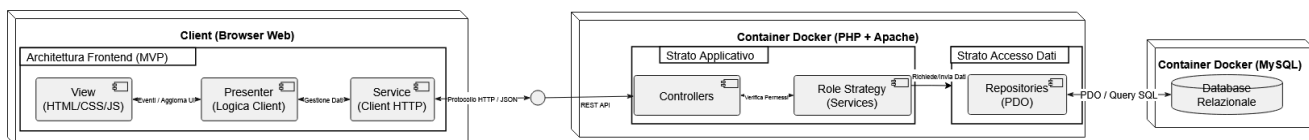
Richiesta HTTP → Controller → Service → Repository → Database

Il backend è organizzato secondo un'architettura a livelli che separa chiaramente la gestione delle richieste, la logica applicativa e l'accesso ai dati. Ogni operazione richiesta dal frontend attraversa una sequenza di componenti specializzati, garantendo modularità, riusabilità e facilità di manutenzione. Il sistema utilizza API REST come unico punto di comunicazione tra client e server.

- Il flusso di elaborazione segue i seguenti passaggi:
- Il frontend invia una richiesta HTTP alle API REST del backend.
- Il componente `api/index.php` riceve la richiesta e la instrada verso il controller appropriato.
- Il Controller interpreta la richiesta, estrae i parametri ricevuti e delega l'operazione al Service corrispondente.
- Il Service applica le regole di business, verifica i permessi dell'utente e valida i dati ricevuti.
- Se necessario, il Service richiede operazioni di persistenza ai Repository.
- I Repository interagiscono con il database tramite il componente dedicato alla gestione della connessione.
- I dati recuperati vengono trasformati in oggetti di dominio (Model) e restituiti al livello superiore.
- Il Service elabora il risultato e costruisce la risposta applicativa.
- Il Controller restituisce una risposta JSON al frontend tramite le API REST.
- Il frontend aggiorna l'interfaccia utente sulla base dei dati ricevuti.

Diagramma delle Componenti

Il diagramma delle componenti rappresenta l'architettura software del sistema evidenziando i principali moduli applicativi e le relazioni che intercorrono tra essi. L'obiettivo del diagramma è mostrare come i diversi livelli dell'applicazione collaborino per realizzare le funzionalità individuate durante le fasi di analisi e progettazione. In particolare vengono evidenziati il frontend organizzato secondo il pattern Model-View-Presenter (MVP), il backend applicativo, i componenti dedicati all'accesso ai dati e il database relazionale utilizzato per la persistenza delle informazioni.



Il diagramma mostra la suddivisione dell'applicazione in tre livelli principali: frontend, backend e persistenza dei dati. Sul lato client il frontend è organizzato secondo il pattern MVP, nel quale le componenti View gestiscono l'interfaccia utente, i Presenter coordinano la logica applicativa e i Service si occupano della comunicazione con il backend.

Il backend è ospitato all'interno di un container Docker basato su PHP e Apache ed è composto da uno strato applicativo e da uno strato di accesso ai dati. I Controller ricevono le richieste provenienti dal frontend tramite API REST, mentre i servizi applicativi implementano le regole di business e la gestione dei ruoli utente.

L'accesso alle informazioni persistenti avviene attraverso il Repository Pattern, che introduce un livello di astrazione tra la logica applicativa e il database. I repository utilizzano PDO per interagire con il database relazionale MySQL, centralizzando le operazioni di lettura e scrittura dei dati.

L'architettura descritta dal diagramma garantisce una netta separazione delle responsabilità tra presentazione, logica applicativa e persistenza dei dati. Questa organizzazione favorisce modularità, manutenibilità ed estendibilità del sistema, risultando coerente con i requisiti funzionali e con le scelte progettuali adottate durante lo sviluppo della piattaforma.

Design Pattern Utilizzati

Durante la progettazione e la realizzazione del sistema sono stati adottati diversi design pattern con l'obiettivo di migliorare la modularità, la manutenibilità e l'estendibilità dell'applicazione. Le scelte effettuate sono state guidate sia dai requisiti funzionali del progetto sia dai vincoli architetturali imposti dall'adozione di un'architettura web basata sul pattern MVP e sull'utilizzo di API REST.

Model-View-Presenter (MVP)

Il pattern architetturale principale adottato nel frontend è il **Model-View-Presenter (MVP)**. In questa architettura le View sono responsabili esclusivamente della gestione dell'interfaccia utente, i Presenter implementano la logica applicativa e coordinano le interazioni tra View e Model, mentre i Model rappresentano i dati e le entità del dominio applicativo. Questa separazione delle responsabilità consente di ridurre l'accoppiamento tra i componenti, semplificare le attività di manutenzione e migliorare la testabilità dell'applicazione.

Repository Pattern

Per la gestione della persistenza dei dati è stato adottato il **Repository Pattern**. Tale pattern introduce un livello di astrazione tra la logica di business e il sistema di memorizzazione dei dati, consentendo alle entità del dominio di interagire con i dati senza dipendere direttamente dalla struttura del database o dai dettagli implementativi delle query.

Nel progetto sono stati previsti repository dedicati alle principali entità del sistema, quali articoli, categorie, tag e commenti. Questa soluzione permette di centralizzare le operazioni di accesso ai dati, migliorando l'organizzazione del codice e facilitando eventuali modifiche future al meccanismo di persistenza.

Factory Method

Per la gestione della creazione delle diverse tipologie di utenti è stato adottato il **Factory Method Pattern**. Il sistema prevede infatti una gerarchia di utenti composta dalle classi Visitor, Editor e Admin, tutte derivate dalla classe astratta User. La responsabilità di istanziare la corretta sottoclasse viene delegata a una factory dedicata, evitando la dispersione della logica di creazione all'interno dell'applicazione. Questa soluzione favorisce il rispetto del principio di responsabilità singola e rende più semplice l'estensione futura del sistema mediante l'introduzione di nuovi ruoli utente.

State Pattern

Per la gestione del ciclo di vita dei commenti è stato adottato lo **State Pattern**. Ogni commento può trovarsi in uno specifico stato di moderazione rappresentato dalle classi PendingState, ApprovedState e RejectedState, tutte derivate dalla classe astratta CommentState.

L'introduzione di questo pattern consente di modellare esplicitamente il processo di approvazione richiesto dal sistema di moderazione. Invece di utilizzare controlli condizionali basati sul valore di uno stato, il comportamento del commento varia in funzione dell'oggetto stato associato. Questa

soluzione migliora la coesione delle classi e rende più semplice l'introduzione di nuovi stati o regole di transizione in future evoluzioni del progetto.

Observer Pattern (Valutato ma non Implementato)

Durante la fase di progettazione è stata considerata la possibilità di utilizzare l'Observer Pattern per gestire l'aggiornamento automatico delle componenti dell'interfaccia in seguito a modifiche dello stato applicativo. Questo pattern risulta particolarmente utile nei sistemi in cui più componenti devono essere notificati in tempo reale quando si verificano cambiamenti ai dati condivisi.

Nel contesto del progetto, tuttavia, tale soluzione non è stata ritenuta necessaria. I requisiti funzionali individuati durante le attività di analisi non prevedono infatti meccanismi di sincronizzazione in tempo reale tra utenti diversi, aggiornamenti automatici delle pagine o funzionalità di notifica istantanea. Gli utenti interagiscono con il sistema attraverso richieste esplicite al backend e le informazioni vengono aggiornate solo in seguito ad azioni dirette dell'utente.

L'introduzione dell'Observer Pattern avrebbe quindi comportato un aumento della complessità architetturale senza apportare benefici significativi rispetto ai requisiti effettivamente richiesti dal sistema. Per questo motivo è stata preferita una soluzione più semplice, basata sulla comunicazione diretta tra le componenti del pattern MVP, nella quale i Presenter aggiornano le View in risposta alle operazioni effettuate dall'utente.

Qualora in future evoluzioni della piattaforma venissero introdotte funzionalità quali aggiornamento automatico dei commenti, notifiche in tempo reale o collaborazione simultanea tra utenti, l'adozione dell'Observer Pattern potrebbe rappresentare una soluzione adeguata per propagare le modifiche tra i componenti interessati in modo disaccoppiato ed estendibile.

Role Strategy Pattern (Valutato ma non Implementato)

Durante la fase di progettazione è stata valutata l'adozione dello **Strategy Pattern** per la gestione dei permessi associati ai diversi ruoli utente della piattaforma. In tale soluzione ogni ruolo avrebbe delegato il proprio comportamento ad una specifica strategia di autorizzazione, consentendo di separare completamente la logica dei permessi dalla struttura delle entità del dominio.

Dopo un'analisi più approfondita del modello applicativo si è scelto di non adottare questa soluzione.

Il sistema prevede infatti una gerarchia di classi basata sull'astrazione User e sulle relative specializzazioni Visitor, Editor e Admin, che rappresentano in modo naturale le differenti responsabilità e capacità operative degli utenti. Attraverso l'ereditarietà è stato possibile associare ad ogni ruolo comportamenti e permessi specifici senza introdurre ulteriori livelli di astrazione.

L'introduzione dello Strategy Pattern avrebbe comportato una duplicazione concettuale delle responsabilità già espresse dalla gerarchia delle classi, aumentando la complessità dell'architettura senza apportare benefici significativi rispetto alle esigenze del progetto. Considerato il numero limitato di ruoli e la semplicità delle regole di autorizzazione richieste, è stata pertanto preferita una soluzione basata su classi astratte e classi derivate, ritenuta più semplice, leggibile e coerente con il modello a oggetti adottato.

Lo Strategy Pattern rimane comunque una possibile evoluzione futura qualora il sistema dovesse supportare un numero maggiore di ruoli o meccanismi di autorizzazione dinamici e configurabili a runtime. In tali scenari la separazione tra entità utente e logica dei permessi potrebbe offrire vantaggi in termini di estendibilità e manutenibilità.

10. Testing

La fase di testing rappresenta una delle attività fondamentali del processo di sviluppo software, poiché consente di verificare che il sistema implementato soddisfi correttamente i requisiti funzionali e non funzionali individuati durante l'analisi e la progettazione. Lo scopo principale del testing è individuare eventuali difetti, anomalie o comportamenti non conformi alle specifiche prima dell'integrazione finale e della consegna del prodotto.

Nel contesto del presente progetto è stato adottato un approccio di verifica progressiva, effettuando controlli durante le diverse fasi di sviluppo al fine di garantire il corretto funzionamento delle componenti software realizzate. Le attività di testing hanno riguardato sia la validazione della logica applicativa sia il controllo della corretta interazione tra le diverse componenti del sistema.

La separazione delle responsabilità introdotta dal pattern Model-View-Presenter (MVP) ha favorito la testabilità dell'applicazione, permettendo di verificare in maniera isolata Presenters, Views e Services. Questo approccio ha consentito di individuare più facilmente eventuali errori implementativi e di semplificare le successive attività di integrazione con il backend REST previsto dall'architettura del sistema.

Testing del Frontend

Per la validazione del frontend è stata adottata una strategia di testing basata sull'utilizzo di componenti mock. L'obiettivo era verificare il comportamento delle interfacce utente e della logica applicativa senza dipendere dalla disponibilità del backend e delle API REST definitive.

A tal fine sono stati realizzati appositi Mock Service e Mock Application in grado di simulare la presenza delle sorgenti dati utilizzate dai Presenter. Questa soluzione ha permesso di testare le funzionalità principali del sistema in maniera isolata, verificando il corretto flusso delle informazioni tra View, Presenter e Service secondo il modello MVP.

Le attività di test hanno interessato tutte le principali funzionalità del sistema, tra cui autenticazione degli utenti, gestione degli articoli, gestione di categorie e tag, moderazione dei commenti, amministrazione degli utenti, ricerca dei contenuti e navigazione tramite categorie e tag. Sono state inoltre verificate le funzionalità della Home Page, compresi i meccanismi di filtro e il ripristino della visualizzazione completa dei contenuti.

Per ogni componente è stata predisposta una specifica pagina HTML di test associata ad una relativa Mock Application, consentendo l'esecuzione indipendente delle verifiche e facilitando l'individuazione di eventuali anomalie prima della fase di integrazione finale con il backend. Questo approccio ha permesso di validare in maniera sistematica l'intero frontend garantendo coerenza con i requisiti funzionali definiti durante l'analisi del progetto.

Struttura dei Mock di Test

None

```
assets/js/test/  
├─ MockCategoryApp.js  
├─ MockCommentManagementApp.js  
├─ MockHomeApp.js  
├─ MockLoginApp.js  
├─ MockPostCreationApp.js  
├─ MockPostDetailApp.js  
├─ MockPostManagementApp.js  
├─ MockSearchApp.js  
├─ MockTagApp.js  
└─ MockUserManagementViewApp.js  
services/MockServices/  
├─ MockAuthService.js  
├─ MockCategoryService.js  
├─ MockCommentService.js  
├─ MockPostService.js  
├─ MockSearchService.js  
├─ MockTagService.js  
└─ MockUserService.js  
pages/TestPages/  
├─ CategoryManagementViewIndex.html  
├─ CommentManagementViewIndex.html  
├─ HomeViewIndex.html  
├─ LoginViewIndex.html  
├─ PostCreationViewIndex.html  
├─ PostDetailViewIndex.html  
├─ PostManagementViewIndex.html  
├─ SearchViewIndex.html  
├─ TagManagementView.html  
└─ UserManagementViewIndex.html
```

Durante la fase di sviluppo del frontend è stata adottata una strategia di testing basata sull'utilizzo di componenti mock al fine di simulare il comportamento del backend e delle API REST non ancora integrate. Questa scelta ha permesso di verificare in maniera isolata il corretto funzionamento delle componenti View e Presenter senza introdurre dipendenze esterne.

L'approccio adottato ha previsto la realizzazione di Service simulati in grado di restituire dati predefiniti e di applicazioni di test dedicate alla validazione delle singole funzionalità del sistema. In questo modo è stato possibile controllare il comportamento dell'interfaccia utente, la corretta gestione degli eventi e il flusso delle informazioni tra Presenter, View e Service secondo il pattern MVP.

I Mock Service hanno simulato le operazioni normalmente svolte dalle API REST, quali autenticazione degli utenti, gestione degli articoli, gestione di categorie e tag, moderazione dei commenti, ricerca dei contenuti e amministrazione degli utenti. Questo ha consentito di eseguire test funzionali completi prima dell'integrazione con il backend definitivo.

L'utilizzo dei mock ha inoltre facilitato l'individuazione precoce di errori logici e problemi di integrazione tra componenti, riducendo la complessità della fase di debugging e consentendo di validare il comportamento dell'applicazione in modo incrementale durante lo sviluppo. Tale approccio risulta particolarmente efficace nei progetti basati su architettura MVP, in quanto favorisce la separazione delle responsabilità e la verificabilità delle singole componenti dell'applicazione.

Tabella dei MockTest eseguiti

Componente	Scenario di Test	Esito
LoginPresenter	Login con credenziali valide	✔ Superato
LoginPresenter	Login con credenziali non valide	✔ Superato
PostPresenter	Creazione articolo	✔ Superato
PostPresenter	Modifica articolo	✔ Superato
PostPresenter	Eliminazione articolo	✔ Superato
CategoryPresenter	Creazione categoria	✔ Superato
CategoryPresenter	Modifica categoria	✔ Superato
CategoryPresenter	Eliminazione categoria	✔ Superato
TagPresenter	Creazione tag	✔ Superato
TagPresenter	Modifica tag	✔ Superato
TagPresenter	Eliminazione tag	✔ Superato
SearchPresenter	Ricerca per parola chiave	✔ Superato
SearchPresenter	Ricerca per categoria	✔ Superato
SearchPresenter	Ricerca per tag	✔ Superato
UserPresenter	Gestione utenti	✔ Superato
CommentPresenter	Creazione commento	✔ Superato

Componente	Scenario di Test	Esito
CommentPresenter	Approvazione commento	✓ Superato
CommentPresenter	Rifiuto commento	✓ Superato
CommentPresenter	Eliminazione commento	✓ Superato
CommentModel + State Pattern	Pending → Approved	✓ Superato
CommentModel + State Pattern	Pending → Rejected	✓ Superato
HomePresenter	Caricamento articoli	✓ Superato

X.Divisione del Lavoro

Per garantire una corretta organizzazione del lavoro e una distribuzione equilibrata delle responsabilità, il progetto è stato sviluppato attraverso la collaborazione di due membri del team. Le attività sono state assegnate in base alle competenze e agli ambiti di interesse dei partecipanti, mantenendo un costante coordinamento durante tutte le fasi del ciclo di sviluppo software. Nelle sezioni seguenti vengono descritte le principali responsabilità assunte da ciascun membro del gruppo e il relativo contributo alla realizzazione del progetto. In quanto team Leader **Daniele Scirpoli** si è occupato molto di adattare le documentazioni parziali del team nel documento finale con l'aggiunta di analisi e introduzioni varie, nonché di organizzare la divisione del lavoro e il coordinamento. Il lavoro è stato diviso in:

Daniele Scirpoli

- Documentazione Finale (scrittura, adattamento dalle Draft Parziali e aggiunta di descrizione e analisi sulla documentazione UML)
- Analisi dei Requisiti
- Specifica dei Requisiti
- Casi d'Uso
- Diagramma delle classi
- Diagramma di Sequenza - Creazione Post
- Architettura progetto (Generale, e in particolare applicazione dell'architettura MVP al frontend)
- Implementazione Interfaccia (HTTP, CSS e Javascript)
- MockTest (Senza Database) dell'architettura MVP implementata

Mattia Cigarini

- Diagramma di Sequenza - Inserimento Commento
- Diagramma di Sequenza - Login
- Diagramma delle Attività
- Diagramma delle Componenti
- Rest API
- Database

Team

Insieme il team si è occupato:

- Organizzazione iniziale del progetto
- Testing
-